Lab – Creating a Custom PHP Application Docker Image

To Begin with the Lab

Summary

In this lab, we learn how to create a custom Docker image for a PHP-based web application using the Apache web server.

The objective is to understand how to package a PHP application into a Docker image and run it inside a container.

We will use the official PHP Docker image with Apache support. Apache will act as the web server, while PHP modules will execute the PHP code.

We will also learn how to copy application files into the Docker image, build the image, create a container, and access the PHP application through a web browser.

Understanding

Understanding PHP Application in Docker

PHP is a server-side programming language used to develop dynamic web applications.

To run a PHP application inside a Docker container, we need:

- **Apache Web Server:** Handles incoming HTTP requests from the browser.
- **PHP Runtime:** Executes PHP code.
- **PHP Application:** Contains files such as index.php and other required libraries.

Docker provides an official PHP image with Apache already configured.

We can use this image as our base image and copy our PHP application files into the appropriate directory.

Lab Steps

Step 1: Open the PHP Application Folder

• Open Visual Studio Code.

• Select **File → Open Folder**.

• Open the folder containing your PHP application.

• Make sure the folder contains the following files:

Note: The index.php file contains the PHP code and HTML used to display the list of courses.

Step 2: Understand the PHP Docker Image

• Open Docker Hub.

• Search for the official PHP image.

• Open the PHP image documentation.

• Navigate to the Tags section.

• Search for an image tag containing apache.

For example:

php:8.2-apache

This image provides:

- PHP 8.2 runtime.
- Apache web server.
- Configuration required to execute PHP applications.

Important: The PHP version is an example. You can use another available version depending on your application requirements.

Step 3: Create a Dockerfile

• Inside your PHP application folder, create a file named Dockerfile.

• Add the following instructions:

```
FROM php:8.2-apache

WORKDIR /var/www/html

COPY . /var/www/html/

EXPOSE 80
CMD ["apache2-foreground"]
```



Explanation:

Instruction	Description
FROM	Specifies the base Docker image.

Instruction	Description
php:8.2-apache	Provides PHP and Apache.
COPY	Copies application files into the image.
.	Represents the current project directory.
/var/www/html/	Apache's default document root in this image.

Note: The index.php file must be available inside /var/www/html/ so Apache can serve the application.

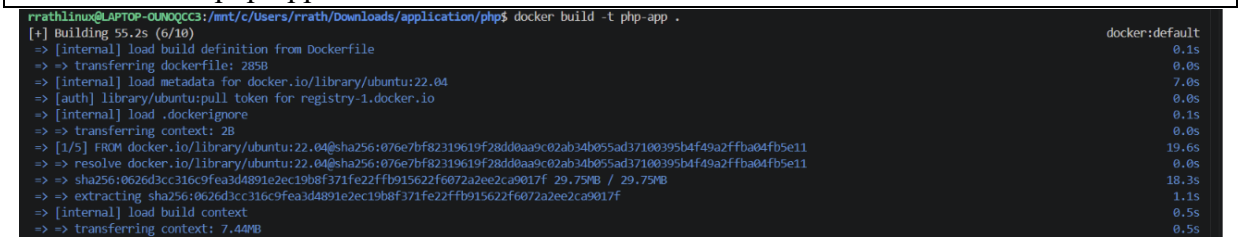
Step 4: Build the Docker Image

• Open the terminal in Visual Studio Code.

• Make sure you are inside the application directory.

• Execute the following command:

```
docker build -t php-app:v1 .
```



Explanation:

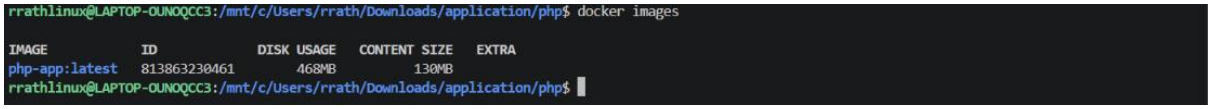
Command	Description
docker build	Builds a Docker image.
-t	Assigns a name and tag to the image.
php-app	Image name.
v1	Image version.
.	Uses the current directory as the build context.

Step 5: Verify the Docker Image

Execute:

```
docker images
```

This command displays all available Docker images.



You should find your newly created image:

REPOSITORY TAG IMAGE ID

php-app v1 xxxxxxxxx

Step 6: Run the Docker Container

Before running the container, make sure port 80 is not already being used by another container.

Execute:

```
docker run -d -p 80:80 --name php-container php-app:v1
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath/Downloads/application/php$ docker run -d -p 8080 --name php-container php-app
9c4b2e058c8c774c8dc4ebac65656fd2e3cc9bca3534e40d2f104356b99f0d08
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath/Downloads/application/php$
```

Explanation:

Option	Description
docker run	Creates and starts a container.
-d	Runs the container in detached mode.
-p 80:80	Maps host port 80 to container port 80.
--name	Assigns a name to the container.
php-app:v1	Specifies the Docker image.

Step 7: Verify the Running Container

Execute:

```
docker ps
```

This command displays currently running containers.

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath/Downloads/application/php$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                                  NAMES
9c4b2e058c8c   php-app   "/bin/sh -c 'apachec…"   43 seconds ago   Up 42 seconds   127.0.0.1:32768->8080/tcp, [::1]:32768->8080/tcp   php-container
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath/Downloads/application/php$
```

To see all containers, including stopped containers:

```
docker ps -a
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath/Downloads/application/php$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED              STATUS              PORTS                                                  NAMES
9c4b2e058c8c   php-app   "/bin/sh -c 'apachec…"   About a minute ago   Up About a minute   127.0.0.1:32768->8080/tcp, [::1]:32768->8080/tcp   php-container
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath/Downloads/application/php$
```

Step 8: Access the PHP Application

• Open your web browser.

• Enter the following address:

http://localhost

• Press Enter.

You should now see your PHP application displaying the list of courses.



Expected Result:

The Apache web server receives the browser request and PHP executes the index.php file.

Your PHP application is now running successfully inside a Docker container.